

Animal Shelter Simulator – Project Design Document

**Owen Lindsey
Professor David Parker
CST-180: Python Programming 1
Grand Canyon University
November 23, 2025**

Important Links

Video: <--- Click there

src code: <--- Click there

Summary

This project establishes an object-oriented design for an animal shelter simulator utilizing **inheritance principles**. The architecture features a base `Animal` class with common attributes (name, age) and behaviors, from which three specialized classes inherit: `Dog`, `Cat`, and `Bird`. Each specialized class extends the base functionality with unique attributes—breed, fur color, and wingspan respectively.

The design includes a `Kenne1` class that demonstrates composition by containing exactly one `Animal` instance at a time. Type identification is achieved through Python's `__class__.__name__` attribute, enabling dynamic animal type reporting.

System Architecture

Data Flow Model

The system follows a three-stage data flow pattern:

Input Stage

The system accepts animal details through constructor parameters. Each animal type requires specific attributes as detailed in the table below:

Animal Type	Required Attributes
Dog	Name (string), Age (integer), Breed (string)
Cat	Name (string), Age (integer), Fur Color (string)
Bird	Name (string), Age (integer), Wingspan (float)

Additionally, the system accepts a selection indicating which animal instance to place within a `Kenne1` container.

Processing Stage

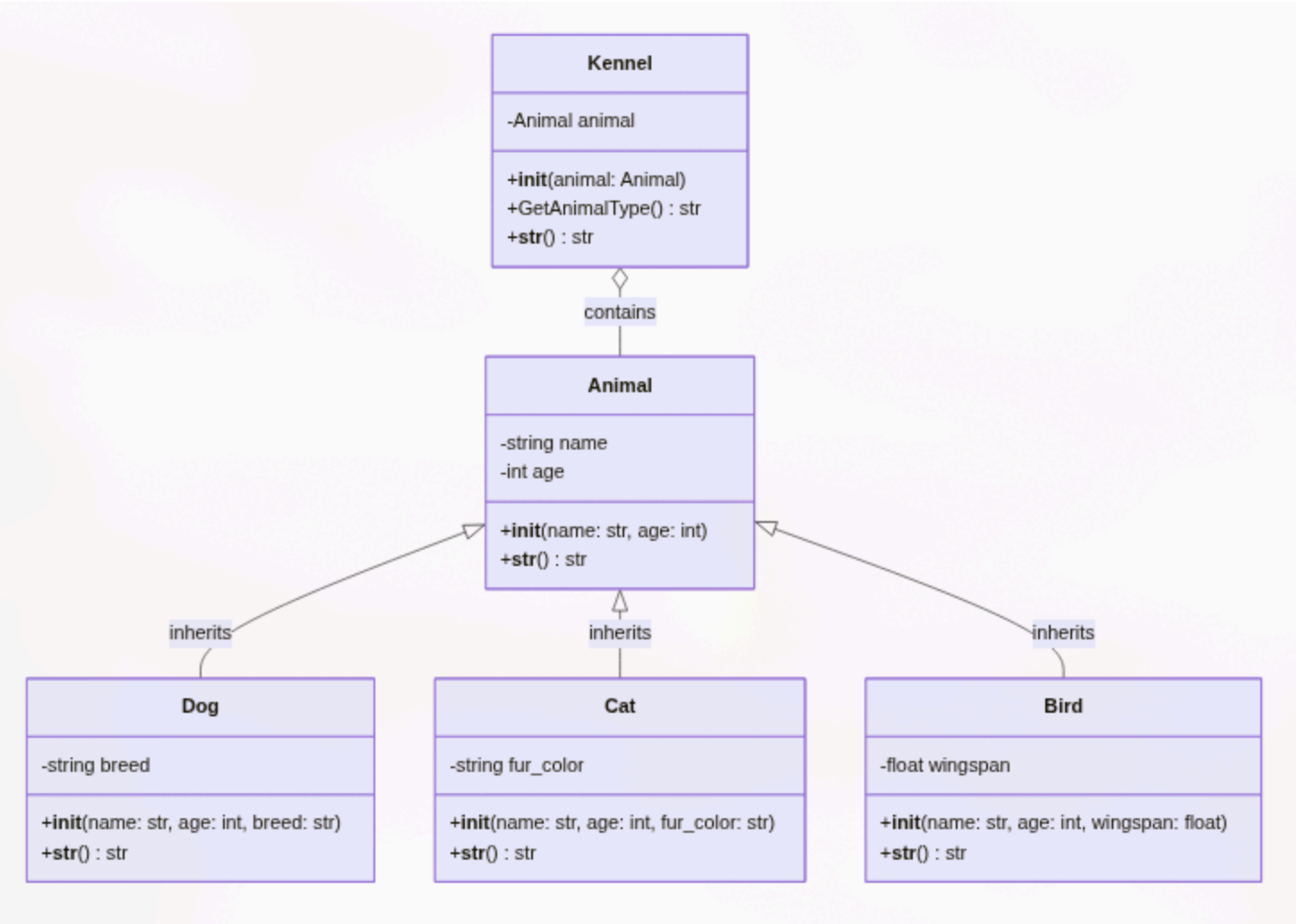
Object construction utilizes overloaded constructors supporting both no-argument (default) and full-argument initialization patterns. The base `Animal` class handles common attributes through inheritance, while specialized classes manage type-specific data. Each class implements a `__str__()` method for human-readable output. The `Kenne1` class employs composition to contain exactly one animal at a time, with dynamic type identification through Python's reflection capabilities (`__class__.__name__`).

Output Stage

The system generates formatted string representations of animal objects and kennel status. The `Kenne1.GetAnimalType()` method returns the class name of the contained animal, providing type information without explicit type checking.

Class Design

UML Class Diagram



Architectural Relationships

The class structure demonstrates two fundamental object-oriented design patterns:

Inheritance Hierarchy

The `Animal` base class serves as the foundation, encapsulating common attributes (name, age) and providing a standard `__str__()` implementation. Three specialized classes—`Dog`, `Cat`, and `Bird`—extend this base, each contributing unique domain-specific attributes: breed, fur color, and wingspan respectively.

Composition Pattern

The `Kennel` class implements a has-a relationship with `Animal`, maintaining a single animal reference at any given time. This design allows the kennel to work polymorphically with any animal subtype while providing runtime type identification through the `GetAnimalType()` method.

Implementation

Python Class Definitions

```
# Base class containing common attributes
class Animal:
    def __init__(self, name: str = "", age: int = 0) -> None:
        self.name = name
        self.age = age

    def __str__(self) -> str:
        return f"Name: {self.name}, Age: {self.age}"

# Dog inherits from Animal and adds breed
class Dog(Animal):
    def __init__(self, name: str = "", age: int = 0, breed: str = "") -> None:
        super().__init__(name, age) # Call parent constructor
        self.breed = breed

    def __str__(self) -> str:
        return f"Dog: Name: {self.name}, Age: {self.age}, Breed: {self.breed}"

# Cat inherits from Animal and adds fur_color
class Cat(Animal):
    def __init__(self, name: str = "", age: int = 0, fur_color: str = "") -> None:
        super().__init__(name, age) # Call parent constructor
        self.fur_color = fur_color

    def __str__(self) -> str:
        return f"Cat: Name: {self.name}, Age: {self.age}, Fur Color: {self.fur_color}"

# Bird inherits from Animal and adds wingspan
class Bird(Animal):
    def __init__(self, name: str = "", age: int = 0, wingspan: float = 0.0) -> None:
        super().__init__(name, age) # Call parent constructor
        self.wingspan = wingspan

    def __str__(self) -> str:
        return f"Bird: Name: {self.name}, Age: {self.age}, Wingspan: {self.wingspan}"

# Kennel can contain any Animal (or subclass)
class Kennel:
    def __init__(self, animal: Animal | None = None) -> None:
        self.animal = animal

    def GetAnimalType(self) -> str:
        return "None" if self.animal is None else self.animal.__class__.__name__

    def __str__(self) -> str:
        return "Kennel(empty)" if self.animal is None else f"Kennel Animal: {self.animal}"
```

Testing and Validation

Test Data Set

The following test instances demonstrate the system's capability to handle diverse animal types:

Instance	Type	Name	Age	Special Attribute	Value
dog1	Dog	Buddy	3	Breed	Labrador
cat1	Cat	Mittens	2	Fur Color	Tabby
bird1	Bird	Sky	1	Wingspan	0.35m

Expected Output

When executed with the test data set, the system produces the following output demonstrating proper inheritance, composition, and type identification:

```
Kennel Animal: Dog: Name: Buddy, Age: 3, Breed: Labrador
Dog

Kennel Animal: Cat: Name: Mittens, Age: 2, Fur Color: Tabby
Cat

Kennel Animal: Bird: Name: Sky, Age: 1, Wingspan: 0.35
Bird
```

The output validates that each animal object correctly inherits base attributes while maintaining specialized characteristics, and the kennel successfully reports the contained animal type.

Application Output:

```
.../Homework/Animal_Shelter main • ) python animal_shelter.py
Kennel Animal: Dog: Name: Buddy, Age: 3, Breed: Labrador
Dog
Kennel Animal: Cat: Name: Mittens, Age: 2, Fur Color: Tabby
Cat
Kennel Animal: Bird: Name: Sky, Age: 1, Wingspan: 0.35
Bird

.../Homework/Animal_Shelter main • )
```

File Output:

```
.../Homework/Animal_Shelter main • ? › python -m unittest test_animal_shelter.py
.....
-----
Ran 28 tests in 0.000s

OK

.../Homework/Animal_Shelter main • ? › python -m unittest discover -v
test_bird_constructor_default (test_animal_shelter.TestBird.test_bird_constructor_default)
Test Bird constructor with default parameters. ... ok
test_bird_constructor_with_parameters (test_animal_shelter.TestBird.test_bird_constructor_with_parameters)
Test Bird constructor with all parameters. ... ok
test_bird_partial_parameters (test_animal_shelter.TestBird.test_bird_partial_parameters)
Test Bird constructor with partial parameters. ... ok
test_bird_str (test_animal_shelter.TestBird.test_bird_str)
Test Bird __str__ method. ... ok
test_cat_constructor_default (test_animal_shelter.TestCat.test_cat_constructor_default)
Test Cat constructor with default parameters. ... ok
test_cat_constructor_with_parameters (test_animal_shelter.TestCat.test_cat_constructor_with_parameters)
Test Cat constructor with all parameters. ... ok
test_cat_partial_parameters (test_animal_shelter.TestCat.test_cat_partial_parameters)
Test Cat constructor with partial parameters. ... ok
test_cat_str (test_animal_shelter.TestCat.test_cat_str)
Test Cat __str__ method. ... ok
test_dog_constructor_default (test_animal_shelter.TestDog.test_dog_constructor_default)
Test Dog constructor with default parameters. ... ok
test_dog_constructor_with_parameters (test_animal_shelter.TestDog.test_dog_constructor_with_parameters)
Test Dog constructor with all parameters. ... ok
test_dog_partial_parameters (test_animal_shelter.TestDog.test_dog_partial_parameters)
Test Dog constructor with partial parameters. ... ok
test_dog_str (test_animal_shelter.TestDog.test_dog_str)
Test Dog __str__ method. ... ok
test_kennel_add_bird (test_animal_shelter.TestKennel.test_kennel_add_bird)
Test adding a Bird to an empty Kennel. ... ok
test_kennel_add_cat (test_animal_shelter.TestKennel.test_kennel_add_cat)
Test adding a Cat to an empty Kennel. ... ok
test_kennel_add_dog (test_animal_shelter.TestKennel.test_kennel_add_dog)
Test adding a Dog to an empty Kennel. ... ok
test_kennel_constructor_empty (test_animal_shelter.TestKennel.test_kennel_constructor_empty)
Test Kennel constructor with no parameters. ... ok
test_kennel_constructor_with_bird (test_animal_shelter.TestKennel.test_kennel_constructor_with_bird)
Test Kennel constructor with a Bird. ... ok
test_kennel_constructor_with_cat (test_animal_shelter.TestKennel.test_kennel_constructor_with_cat)
Test Kennel constructor with a Cat. ... ok
test_kennel_constructor_with_dog (test_animal_shelter.TestKennel.test_kennel_constructor_with_dog)
Test Kennel constructor with a Dog. ... ok
test_kennel_get_animal_type_bird (test_animal_shelter.TestKennel.test_kennel_get_animal_type_bird)
Test GetAnimalType method with a Bird. ... ok
test_kennel_get_animal_type_cat (test_animal_shelter.TestKennel.test_kennel_get_animal_type_cat)
Test GetAnimalType method with a Cat. ... ok
test_kennel_get_animal_type_dog (test_animal_shelter.TestKennel.test_kennel_get_animal_type_dog)
Test GetAnimalType method with a Dog. ... ok
test_kennel_get_animal_type_empty (test_animal_shelter.TestKennel.test_kennel_get_animal_type_empty)
Test GetAnimalType method with empty kennel. ... ok
test_kennel_replace_animal (test_animal_shelter.TestKennel.test_kennel_replace_animal)
Test replacing an animal in the Kennel (only one animal at a time). ... ok
test_kennel_str_empty (test_animal_shelter.TestKennel.test_kennel_str_empty)
Test Kennel __str__ method when empty. ... ok
test_kennel_str_with_bird (test_animal_shelter.TestKennel.test_kennel_str_with_bird)
Test Kennel __str__ method with a Bird. ... ok
test_kennel_str_with_cat (test_animal_shelter.TestKennel.test_kennel_str_with_cat)
Test Kennel __str__ method with a Cat. ... ok
test_kennel_str_with_dog (test_animal_shelter.TestKennel.test_kennel_str_with_dog)
Test Kennel __str__ method with a Dog. ... ok

-----
Ran 28 tests in 0.000s

OK
```


Test Output:

```
Neo-tree
/run/media/omniv/T7/GCU_SWE_2023-2025
> AIT-104-Data-mining-machine-learning
> CST-150-C#1
v CST-180-Python
  v Homework
    v Animal_Shelter
      v __pycache__
        Animal_Shelter_Pseudocode.md
        Animal_Shelter_Pseudocode.pdf
        animal_shelter.md
        animal_shelter.py
        animal_shelter_p1.py
        animal_shelter_pt2.md
        animal_shelter_pt2.pdf
        test_animal_shelter.py
      PopulationSimulation
      TheologicalFoundations
      War Card Game
      input-output
      NeoVimPractice.py
      (1 hidden item)
    Photos
    Templates
    Zybook_notes
    class notes
    plugins
    snippets
    themes
    Pasted image 20251106094822.png
    Pasted image 20251106105418.png
    Pasted image 20251106145000.png?
    Pasted image 20251106145210.png?
    README.md
    app.json
    appearance.json
    community-plugins.json
    core-plugins.json
    graph.json
    workspace.json
    (1 hidden item)
  CST-201-algorithms-data-structures
  CST-239-Java2
  CST-250-C#2
  CST-321-Operating-system-fundamentals
  CST-326-Written-Verbal-Communication
  CST-339-Java3
  CST-345-Database-Design
  CST-350-C#3
  CST-391-Web_dev
  CST-407-Application-Security-Fundamentals
  SummerPractice
  LICENSE
  README.md
  orders-response.json
  swagger-spec.json
  (4 hidden items)
```

Dalbey, J. (2003). *Pseudocode Standard*. California Polytechnic State University. Retrieved from https://users.csc.calpoly.edu/~jdalbey/SWE/pdl_std.html